

Нагрузочное тестирование проверка вашего кода на прочность

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?



Преподаватель



Евгений Волобуев

Full Stack C# .NET Developer

- Ведущий программист компании Simbirsoft
- Более 6 лет в сфере промышленной и коммерческой разработки
- Опыт разработки высоконагруженных и отказоустойчивых систем
- В составе команды создаю мобильный интернет банк для стран латинской америки

 @e_volobuev

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **OTUS C#-
in-depth-2025-08**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Карта курса



СТАРТ

Управление памятью
на уровне ядра

Асинхронность и
многопоточность
для экспертов

Внутренние
механизмы CLR

Профилирование и
экстремальная
оптимизация

Современные
парадигмы и
инструменты эксперта

Проектный
модуль

ФИНИШ



Цели вебинара

К концу занятия вы сможете

1. Планировать и проводить нагрузочное тестирование для сетевых сервисов;
2. Использовать NBomber для генерации нагрузки;
3. Анализировать результаты тестов: RPS (запросы в секунду), время отклика (latency), количество ошибок;
4. Определять максимальную пропускную способность и точки отказа приложения.



Цели вебинара

К концу занятия вы сможете

Проводить нагрузочное тестирование

Применять NBomber для генерации нагрузки

Анализировать результаты тестов

Смысл

Для чего вам это уметь

Определять максимальную пропускную способность и точки отказа приложения



Маршрут вебинара

Виды тестирования (Load vs Stress vs Soak)

Метрики (RPS, Latency, Error Rate)

NBomber (архитектура + примеры)

Практический пример TCP-кэша

OpenTelemetry + dotnet-counters диагностика

Рефлексия



Тестовый пример

Тестовый пример можно скачать по ссылке:

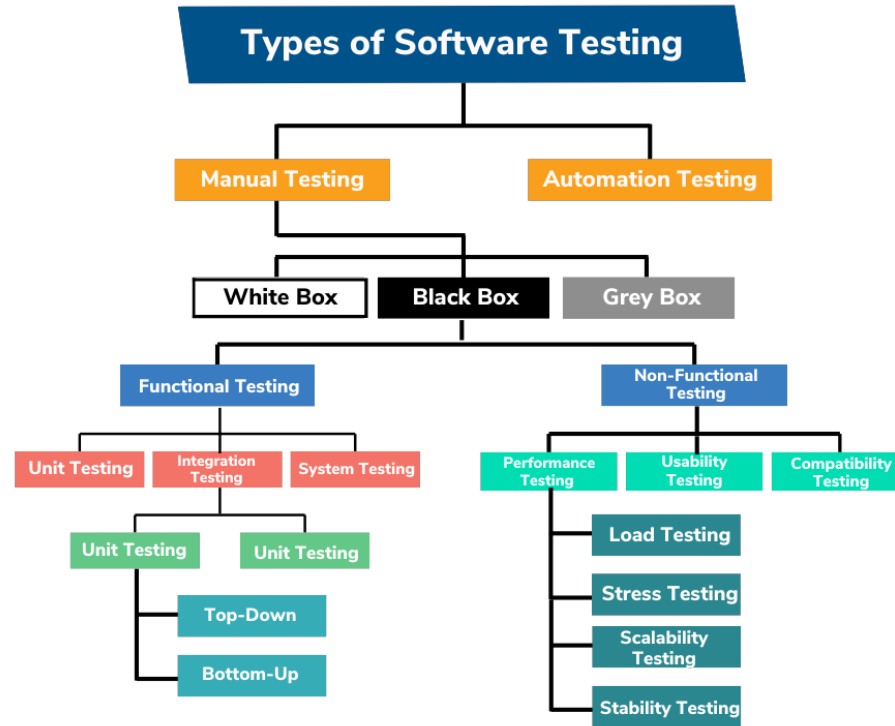
<https://github.com/Dzitskiy/NbomberDemo>

https://github.com/Dzitskiy/k6_loadtest

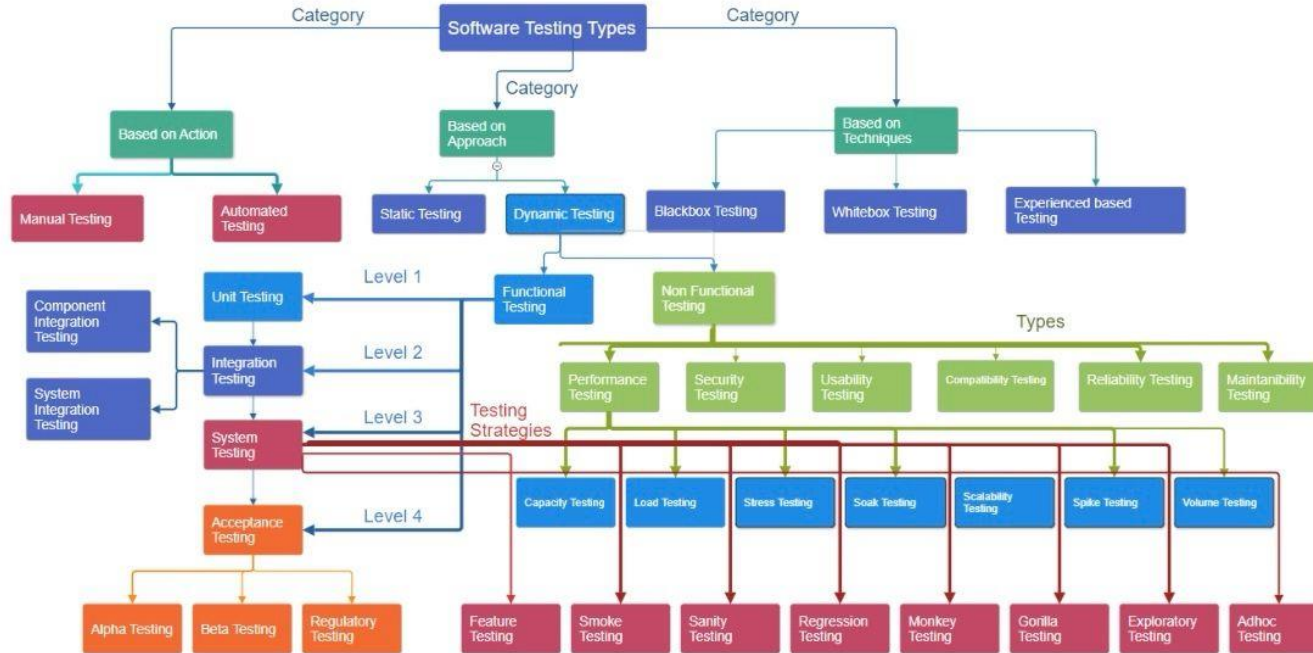


Виды тестирования (Load vs Stress vs Soak)

Виды тестирования



Виды тестирования



https://www.linkedin.com/posts/atika-farooq-151686183_software-testing-hierarchy-at-a-glance-this-activity-7202490692378464257-1Ozp/



Зачем это нужно?

Проблема:

«У меня на локальной машине всё летает, а в продакшене падает под нагрузкой».

Цель нагрузки:

Не найти функциональные баги, а оценить поведение системы при реальной или запланированной нагрузке.



Зачем это нужно?

Проблема:

«У меня на локальной машине всё летает, а в продакшене падает под нагрузкой».

Цель нагрузки:

Не найти функциональные баги, а оценить поведение системы при реальной или запланированной нагрузке.

Ключевые вопросы:

- Сколько пользователей/запросов выдержит мой сервис?
- Каково время отклика (latency) при пиковой нагрузке?
- Где узкие места (bottlenecks): ЦПУ, память, потоки, БД, сеть?
- Как система деградирует при перегрузке?



Performance Testing

Performance Testing (Тестирование производительности) — это общее название для нефункционального тестирования, которое оценивает, насколько хорошо система или приложение работает с точки зрения скорости, отзывчивости, стабильности, масштабируемости и эффективности использования ресурсов при различных условиях и нагрузках.

Основная цель — не найти дефекты в бизнес-логике, а обеспечить и подтвердить требуемый уровень качества работы системы (Performance Quality Attributes) и выявить проблемы, которые необходимо устранить до выпуска продукта.



Тестирование производительности

Тестирование производительности — это стратегическая деятельность, направленная на то, чтобы система не только работала (*functional*), но и работала хорошо, быстро и стабильно (*non-functional*) как при обычных условиях, так и в периоды стресса или роста.

Это критически важный процесс для обеспечения удовлетворенности пользователей, предотвращения финансовых потерь и поддержания деловой репутации.

Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- Объемное тестирование (Volume / Capacity Testing)
- Тестирование пиковой нагрузки (Spike Testing)
- Тестирование масштабируемости (Scalability Testing)
- Конфигурационное тестирование (Configuration Testing)



Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)

Что: Проверка системы под ожидаемой нагрузкой (например, 1000 одновременных пользователей).

Зачем: Убедиться, что система работает стабильно и отвечает в рамках SLA (Service Level Agreement — соглашение об уровне обслуживания) при нормальных условиях.

- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- Объемное тестирование (Volume / Capacity Testing)
- Тестирование пиковой нагрузки (Spike Testing)
- Тестирование масштабируемости (Scalability Testing)
- Конфигурационное тестирование (Configuration Testing)

Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- **Стресс-тестирование (Stress Testing)**

Что: Постепенное увеличение нагрузки за пределы нормальных условий, чтобы найти "точку разрыва" системы. Например, увеличение с 1000 до 5000 пользователей.

Зачем: Определить, как система ведет себя в экстремальных условиях, как и когда она деградирует (падает, зависает), и как восстанавливается после снятия нагрузки. Помогает понять "запас прочности".

- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- Объемное тестирование (Volume / Capacity Testing)
- Тестирование пиковой нагрузки (Spike Testing)
- Тестирование масштабируемости (Scalability Testing)
- Конфигурационное тестирование (Configuration Testing)

Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)

Что: Длительное (часы, сутки) тестирование под средней или высокой нагрузкой.

Зачем: Обнаружить проблемы, которые проявляются со временем: утечки памяти (memory leaks), фрагментация, переполнение логов, деградация производительности после долгой работы.

- Объемное тестирование (Volume / Capacity Testing)
- Тестирование пиковой нагрузки (Spike Testing)
- Тестирование масштабируемости (Scalability Testing)
- Конфигурационное тестирование (Configuration Testing)



Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- **Объемное тестирование (Volume / Capacity Testing)**

Что: Работа с большими объемами данных. Например, загрузка в базу данных 10 миллионов записей.

Зачем: Проверить, как система справляется с ростом объема данных (не путать с количеством пользователей). Влияет ли это на скорость поиска, время отклика, потребление дискового пространства.

- Тестирование пиковой нагрузки (Spike Testing)
- Тестирование масштабируемости (Scalability Testing)
- Конфигурационное тестирование (Configuration Testing)

Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- Объемное тестирование (Volume / Capacity Testing)
- **Тестирование пиковой нагрузки (Spike Testing)**

Что: Резкое, кратковременное увеличение нагрузки (скачок). Например, мгновенный рост с 100 до 2000 пользователей, а затем сброс.

Зачем: Проверить, как система реагирует на внезапные всплески активности (например, при запуске распродажи или после публикации новости).

- Тестирование масштабируемости (Scalability Testing)
- Конфигурационное тестирование (Configuration Testing)

Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- Объемное тестирование (Volume / Capacity Testing)
- Тестирование пиковой нагрузки (Spike Testing)
- **Тестирование масштабируемости (Scalability Testing)**

Что: Проверка способности системы увеличивать производительность при добавлении ресурсов (больше серверов, более мощное железо).

Зачем: Определить, как можно наращивать мощность системы для будущего роста и является ли это линейным (удвоение серверов = удвоение производительности).

- Конфигурационное тестирование (Configuration Testing)

Основные виды тестирования производительности

- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)
- Объемное тестирование (Volume / Capacity Testing)
- Тестирование пиковой нагрузки (Spike Testing)
- Тестирование масштабируемости (Scalability Testing)
- **Конфигурационное тестирование (Configuration Testing)**

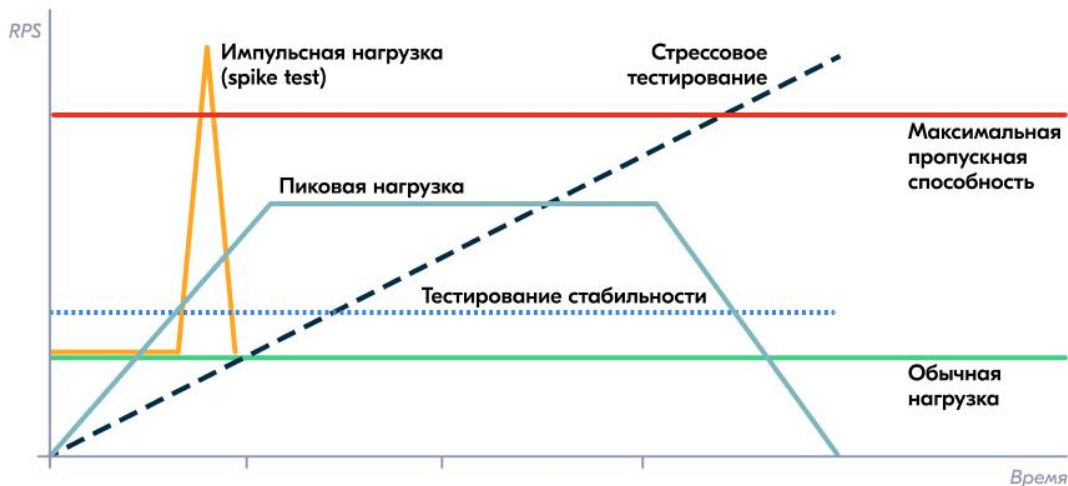
Что: Тестирование производительности при разных конфигурациях ПО и железа (разные версии БД, настройки веб-сервера, объем оперативной памяти).

Зачем: Найти оптимальную конфигурацию для максимальной производительности.

Тестирования производительности



- Нагрузочное тестирование (Load Testing)
- Стресс-тестирование (Stress Testing)
- Тестирование на выносливость/стабильность (Soak / Endurance Testing)



Нагрузочное тестирование

Нагрузочное тестирование (Load Testing) — это один из ключевых видов нефункционального тестирования, целью которого является оценка поведения системы при ожидаемой (или близкой к ней) нагрузке.

Это процесс, когда на приложение (веб-сайт, мобильное приложение, сервер API) симулируют работу множества реальных пользователей, чтобы понять:

- Как быстро оно отвечает;
- Насколько стабильно работает;
- Где находятся его "узкие места" (bottlenecks) до того, как приложение выйдет в продакшен и с этими проблемами столкнутся реальные клиенты.



Основные цели нагрузочного тестирования

- **Определение производительности:**

Измерение времени отклика ключевых операций (например, загрузка страницы, выполнение поиска, оформление заказа) под нагрузкой.

- **Поиск "узких мест":**

Выявление компонентов, которые замедляют работу системы (медленная база данных, недостаточно мощный сервер, неэффективный код).

- **Определение пропускной способности:**

Сколько пользователей/запросов в секунду система может обработать, сохраняя приемлемое время отклика.

- **Проверка стабильности:**

Работает ли система стабильно под длительной нагрузкой (нет утечек памяти, ошибок, падений).

- **Валидация инфраструктуры:**

Достаточно ли у нас серверов, мощности процессора, оперативной памяти, пропускной способности сети?



Популярные инструменты

- [Apache JMeter](#):

Самый популярный бесплатный инструмент с графическим интерфейсом.

- [k6](#):

Современный инструмент с написанием сценариев на JavaScript, ориентированный на разработчиков.

- [Gatling](#):

Высокопроизводительный фреймворк на Scala.

- **LoadRunner**:

Мощный коммерческий инструмент от Micro Focus.

- **Облачные сервисы:**

Loader.io, BlazeMeter, Azure Load Testing (управляют инфраструктурой за вас).



Вопросы



если есть вопросы



если вопросов нет

Метрики (RPS, Latency, Error Rate)

Ключевые метрики

- **Время отклика (Response Time):**

Время от отправки запроса до получения полного ответа. Часто измеряют перцентили (95-й, 99-й перцентиль). Норма для продакшена — 200–500 мс; включает задержку, обработку и передачу.

- **Задержка (Latency):**

Время сетевой передачи данных, измеряется пингом; влияет на общее время отклика.

- **Перцентили (P95/P99):**

Время отклика для 95% или 99% запросов, выявляет редкие пики нагрузки.

- **Количество ошибок (Error Rate):**

Процент запросов, завершившихся с ошибкой под нагрузкой.

- **Количество одновременных пользователей:**

Виртуальных (VU) или реальных.

- **Использование ресурсов:**

Загрузка процессора (CPU), оперативной памяти (RAM), дискового ввода-вывода (Disk I/O), сети (Network I/O) на серверах.

- **Время восстановления (Recovery Time):**

Как быстро система приходит в норму после снятия пиковой нагрузки или сбоя.



RPS (Requests Per Second)

Что это

- Количество успешных HTTP-запросов, которые система обрабатывает за одну секунду
- Показатель пропускной способности (throughput) системы

На что смотреть

- *Абсолютное значение*: Сколько запросов/сек выдерживает система
- *Стабильность*: Сохраняется ли RPS при длительной нагрузке
- *Точка насыщения*: При каком количестве пользователей RPS перестает расти

Важные паттерны

- *P99 >> P95*: Проблемы с отдельными запросами (медленные БД-запросы, блокировки)
- *Равномерный рост P95 и P99*: Система равномерно деградирует
- *P99 стабилен при росте нагрузки*: Система масштабируется хорошо

Практическая интерпретация

- *Хорошо*: RPS стабильно высокий при росте нагрузки
- *Плохо*: RPS падает при увеличении числа пользователей

Latency (P95, P99)

Что это

- **P95**: 95% запросов выполняются быстрее этого значения
- **P99**: 99% запросов выполняются быстрее этого значения
- P99 показывает "хвост" распределения - наихудшие 1% случаев

Критические значения (пользовательский опыт)

- **< 100 мс**: Отлично (незаметно)
- **100-300 мс**: Хорошо
- **300-1000 мс**: Приемлемо
- **> 1000 мс**: Проблема (пользователи замечают)
- **> 3000 мс**: Критично (пользователи уходят)

Важные паттерны

- **P99 >> P95**: Проблемы с отдельными запросами (медленные БД-запросы, блокировки)
- **Равномерный рост P95 и P99**: Система равномерно деградирует
- **P99 стабилен при росте нагрузки**: Система масштабируется хорошо

Error Rate

Что учитывать

- HTTP ошибки (4xx, 5xx)
- Таймауты соединений
- Несоответствие ожидаемому результату (failed checks)

Допустимые значения

- *< 0.1%: Отлично*
- *0.1%-1%: Требуется исследования*
- *> 1%: Критично (необходимо немедленное исправление)*
- *> 5%: Система неработоспособна*

Важные паттерны

- *P99 >> P95*: Проблемы с отдельными запросами (медленные БД-запросы, блокировки)
- *Равномерный рост P95 и P99*: Система равномерно деградирует
- *P99 стабилен при росте нагрузки*: Система масштабируется хорошо

Взаимосвязь метрик (Анализ паттернов)

Сценарий 1: Здоровая система

- **RPS:** ↑ растет пропорционально нагрузке
- **P95/P99:** → остаются в SLA
- **Error Rate:** → < 0.1%

Вывод: Система масштабируется

Сценарий 2: Проблемы с БД/бэкендом

- **RPS:** → достигает плато
- **P95:** ↑ растет умеренно
- **P99:** ↑↑ растет значительно (хвост)
- **Error Rate:** ↑ появляются таймауты

Вывод: Узкое место в обработке запросов

Сценарий 3: Проблемы с памятью/утечки

- **RPS:** ↓ падает со временем
- **P95/P99:** ↑↑ постоянно растут
- **Error Rate:** ↑ растет к концу теста

Вывод: Проблемы при длительной нагрузке

Что фиксировать в отчете

Базовые показатели:

- Максимальный достигнутый RPS
- P95/P99 при пиковой нагрузке
- Общий Error Rate

Точки перелома:

- При какой нагрузке Error Rate превысил 1%
- При каком RPS начался рост latency

Рекомендации:

- Целевые значения для оптимизации
- Приоритетность исправлений



Практический пример

Выполним нагрузочное тестирование WebAPI с помощью k6:

- Установите k6
- Сохраните код в файл loadtest.js (https://github.com/Dzitskiy/k6_loadtest)
- Запустите тесты
- Проанализируйте результаты

Метрика	На что смотреть	Что означает
http_req_duration	Значения p(95) и p(99) (95-й и 99-й процентиля).	Время ответа сервера. Например, p(95)=850ms значит, что 95% запросов обработались быстрее 850 мс.
http_req_failed	Процент (rate). Должен быть близок к 0.	Доля неудачных запросов (ошибки 4xx/5xx, таймауты сети).
checks	Процент успешных проверок (checks_succeeded).	Надежность бизнес-логики (например, корректность тела ответа).
http_reqs и iterations	Общее количество и скорость (http_reqs/s).	Пропускная способность системы (throughput).
vus	Максимальное количество (vus_max).	Пиковое число одновременных виртуальных пользователей.

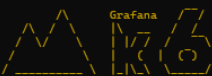
Практический пример

```
import http from 'k6/http';
import { check, sleep, group } from 'k6';
import { Trend, Rate, Counter } from 'k6/metrics';

// 1. КОНФИГУРАЦИЯ И МЕТРИКИ
// УКАЖИТЕ АДРЕС ВАШЕГО API (возможно, потребуется изменить пор
const BASE_URL = 'http://dzitskiy.ru:5000';

// Объявление пользовательских метрик для глубокого анализа[cit
const requestDuration = new Trend('http_req_duration_custom');
const requestFailureRate = new Rate('http_req_failed_custom');
const successfulChecks = new Counter('checks_succeeded');

// 2. ОСНОВНЫЕ ОПЦИИ ТЕСТА (конфигурация сценариев)
export const options = {
  // Пороги производительности (performance thresholds). Тест
  thresholds: {
    'http_req_duration{scenario:smoke}': ['p(95) < 1000'],
    'http_req_duration{scenario:load}': ['p(95) < 2000'],
    'http_req_failed': ['rate < 0.05'],
    'checks_succeeded': ['count > 0']
  },
  // Определение сценариев[citation:9]
  scenarios: {
    // СЦЕНАРИЙ 1: Дымовое тестирование (проверка работоспо
    smoke: {
      executor: 'shared-iterations',
      vus: 2, // 2 виртуальных пользо
      iterations: 5, // Всего 5 итераций (вы
      maxDuration: '1m',
      tags: { scenario: 'smoke' }, // Теги для фильтрации
    },
  },
};
```



```
execution: local
script: loadtest.js
output: json (test_results.json)

scenarios: (100.00%) 3 scenarios, 50 max VUs, 30s max duration (incl. graceful stop):
  * load: Up to 10 looping VUs for 1m50s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
  * smoke: 5 iterations shared among 2 VUs (maxDuration: 1m50s, gracefulStop: 30s)
  * stress: Up to 40 looping VUs for 2m30s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

INFO[0000] Настройка тестового окружения... source=console
INFO[0150] Тест запущен в: 2025-12-04T02:17:10.026Z source=console
INFO[0150] Очистка тестового окружения... source=console

THRESHOLDS

checks_succeeded
✓ 'count > 0' count=4902

http_req_duration{scenario:load}
✓ 'p(95) < 2000' p(95)=331.64ms

http_req_duration{scenario:smoke}
✓ 'p(95) < 1000' p(95)=295.1ms

http_req_failed
✓ 'rate < 0.05' rate=0.00%

TOTAL RESULTS

checks_total.....: 9804 65.667894/s
checks_succeeded.....: 100.00% 9804 out of 9804
checks_failed.....: 0.00% 0 out of 9804

HTTP
http_req_duration.....: avg=287.54ms min=186.18ms med=269.24ms max=980.44ms p(90)=357.4ms p(95)=407.74ms
{ scenario:load }.....: avg=267.3ms min=222.39ms med=259.93ms max=730.92ms p(90)=307.63ms p(95)=331.64ms
{ scenario:smoke }.....: avg=272.96ms min=257.36ms med=265.8ms max=297.66ms p(90)=292.54ms p(95)=295.1ms
http_req_duration_custom.....: avg=287.543461ms min=186.1851ms med=269.2419s max=980.4473 p(90)=357.40815 p(95)=407.740315
http_req_failed.....: 0.00% 0 out of 4902
http_req_failed_custom.....: 0.00% 0 out of 4902
http_reqs.....: 4902 32.533947/s

EXECUTION
iteration_duration.....: avg=788.85ms min=687.55ms med=770.27ms max=1.48s p(90)=858.77ms p(95)=908.89ms
iterations.....: 4902 32.533947/s
vus.....: 1 min=1 max=30
vus_max.....: 50 min=50 max=50
```

LIVE

Вопросы



если есть вопросы



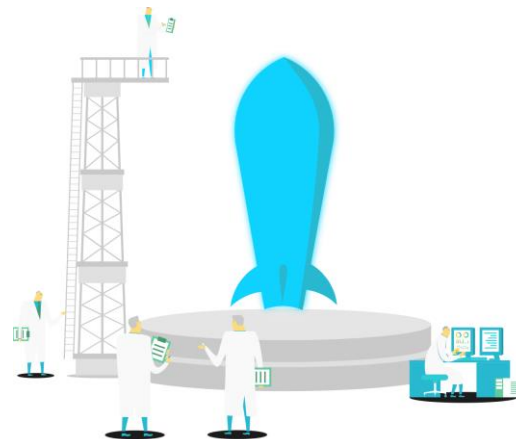
если вопросов нет

NBomber.

Тесты как код

Почему NBomber?

- Нативный для .NET (нет привязки к JS/Python).
- Полноценный фреймворк, а не просто генератор запросов.
- Философия «Tests as Code»: сценарии пишутся на C#/F# — это гибко, типобезопасно и интегрируется в ваш процесс разработки.



NBomber

NBomber — это фреймворк нагрузочного тестирования для .NET, который позволяет описывать сценарии на C#/F#, управлять профилями нагрузки (RPS/конкурентность) и получать детальную статистику (RPS, latency, ошибки) в консоль, HTML и текстовых отчётах.

- Кроссплатформенный open-source фреймворк нагрузочного тестирования для .NET (C#/F#). Тесты описываются как обычный код, нет DSL или UI-скриптов.
- Поддерживает любые протоколы (HTTP, WebSockets, MQTT, gRPC, TCP и т.п.), поскольку сценарий — это произвольный C#-код, в котором разработчик сам выполняет запросы и оборачивает их в шаги.
- Управляет потоками, планированием нагрузки и сбором метрик, предоставляя подробные отчёты по RPS, latency (p50, p75, p95, p99), ошибкам, data transfer и др.
- Поставляется как NuGet-пакет NBomber, можно запускать как библиотеку из своего тестового кода или через CLI/runner, есть поддержка distributed/cluster-режима.



dotnet add package NBomber



```
using NBomber.NBomberRunner;
```

```
NBomberRunner  
    .RegisterScenarios(/* your scenarios */)   
    .RunTest();
```

Базовая модель NBomber

- **Step (Шаг)** — атомарная операция (например, GET или SET команда к кэшу).
- **Scenario (Сценарий)** — цепочка шагов, которая выполняется виртуальными пользователями.
- **Simulation (Симуляция)** — стратегия «впрыска» пользователей (разгон, постоянная нагрузка, пики).

Архитектурные понятия (сценарий, шаги, симуляции)

- **Сценарий (Scenario)** — модель поведения виртуального пользователя: последовательность действий, которую каждый виртуальный юзер повторяет под нагрузкой.
- **Шаги (Step)** — атомарные действия внутри сценария (HTTP-запрос, запрос в БД, publish в очередь, комплексная операция). Каждый шаг возвращает Response.Ok или Response.Fail и измеряется отдельно.
- Профиль нагрузки задаётся через *WithLoadSimulations*, где используются симуляции:
- *Simulation.Inject* — фиксированный или изменяющийся RPS (rate/interval/during).
- *Simulation.KeepConstant* — постоянное количество одновременных виртуальных пользователей.



Построение сценария (Scenario)

В NBomber основные методы настройки нагрузочной симуляции — это *Simulation.Inject* и *Simulation.KeepConstant*.

Они контролируют, как создаётся нагрузка во времени.

Фаза разогрева (Warm-up):

Simulation.Inject(rate: 5, interval: TimeSpan.FromSeconds(1), during: TimeSpan.FromSeconds(30))
— плавный подъем нагрузки для «прогрева» JIT, пулов соединений, кэшей.

Фаза основной нагрузки:

Simulation.KeepConstant(copies: 100, during: TimeSpan.FromMinutes(2)) — 100 виртуальных пользователей постоянно выполняют сценарий. Или *Simulation.RampPerSec* для поиска предела.

Композиция: Сценарий может смешивать шаги в разных пропорциях (например, 70% GET и 30% SET запросов).



Simulation.Inject

- Используется для генерации фиксированного количества запросов в секунду (RPS) в течение указанного времени.
- Позволяет задать: сколько запросов в секунду, интервал и длительность.
- Если время ответа растёт, запросы будут ставиться в очередь — нагрузка фиксирована и не адаптируется.

Синтаксис:

```
Simulation.Inject(  
    rate: 100,                // количество запросов в секунду  
    interval: TimeSpan.FromSeconds(1), // интервал измерения (обычно 1 секунда)  
    during: TimeSpan.FromMinutes(5)    // длительность нагрузки  
)
```


Simulation.KeepConstant

- Поддерживает фиксированное число параллельных пользователей (копий сценария) постоянно.
- Генерирует столько одновременных запросов, сколько указано в `copies`.
- Размер нагрузки адаптируется под скорость отклика: если `latency` растёт, реальный RPS падает, но количество параллельных пользователей остаётся постоянным.
- Моделирует реальную ситуацию с фиксированным числом клиентов.

Синтаксис:

```
Simulation.KeepConstant(  
    copies: 50,                // число параллельных пользователей  
    during: TimeSpan.FromMinutes(10) // длительность нагрузки  
)
```

Отличия и когда использовать

Метод	Что задаёт	Поведение при росте latency	Когда использовать
Simulation.Inject	RPS (запросы в секунду)	Загрузку поддерживает фиксированной, очередь растёт	Для теста максимальной пропускной способности и стресс-тестов
Simulation.KeepConstant	Количество одновременных пользователей	Количество пользователей фиксировано, RPS может падать при росте latency	Для моделирования реальных пользователей и устойчивости системы

Практические советы:

- Для warm-up часто используют Simulation.Inject с низким rate.
- Для основной нагрузки — Simulation.KeepConstant для имитации реальных пользователей.
- Можно комбинировать несколько симуляций:

```
.WithLoadSimulations(  
    Simulation.Inject(100, TimeSpan.FromSeconds(1), TimeSpan.FromMinutes(2)), // warm-up  
    Simulation.KeepConstant(50, TimeSpan.FromMinutes(8)) // steady load  
)
```

Такая настройка даст плавный разогрев и стабильный нагрузочный тест.



Практический пример

```
using NBomber.CSharp;
using NBomber.Contracts;

var step = Step.Create("delay_step", async context =>
{
    await Task.Delay(TimeSpan.FromMilliseconds(
        Random.Shared.Next(100, 300)));
    return Response.Ok();
});

var scenario = Scenario.Create("simple_scenario", step)
    .WithLoadSimulations(
        Simulation.Inject(rate: 10,
            interval: TimeSpan.FromSeconds(1),
            during: TimeSpan.FromSeconds(30))
    );

NBomberRunner
    .RegisterScenarios(scenario)
    .Run();
```



```
18:41:44 [INF] NBomber "6.0.1" started a new session: "2025-12-04_15.41.06_session_410410d4"
18:41:44 [INF] NBomber started as single node
18:41:44 [INF] License validation...
18:41:44 [WRN] THIS VERSION IS FREE ONLY FOR PERSONAL USE. You can't use it for an organization.
18:41:44 [INF] Reports folder: "D:\Projs\OTUS\NBomberDemo\NBomberDemo\bin\Debug\net8.0\reports\2025-12-04_15.41.06_session_410410d4"
18:41:44 [INF] Plugins: no plugins were loaded
18:41:44 [INF] Reporting sinks: no reporting sinks were loaded
18:41:44 [INF] Starting init...
18:41:44 [INF] Target scenarios: "http_test"
18:41:44 [INF] Init finished
18:41:44 [INF] Starting bombing...
18:42:46 [INF] Waiting on scenarios completion...
18:43:05 [WRN] Stopping test early: "Stopping test because of too many fails. Scenario 'http_test'."
18:43:08 [INF] Calculating final statistics...
```

test suite: nbomber_default_test_suite_name
test name: nbomber_default_test_name
session id: 2025-12-04_15.41.06_session_410410d4

scenario: http_test

- ok count: 0
- fail count: 5100
- all data: 0 MB
- duration: 00:00:51

load simulations:

- inject, rate: 100, interval: 00:00:01, during: 00:01:00

step	ok stats
name	global information
request count	all = 5100, ok = 0, RPS = 0
latency (ms)	min = 0, mean = 0, max = 0, StdDev = 0
latency percentile (ms)	p50 = 0, p75 = 0, p95 = 0, p99 = 0

step	failures stats
name	global information
request count	all = 5100, fail = 5100, RPS = 100
latency (ms)	min = 171.93, mean = 451.54, max = 2600.17, StdDev = 299.04
latency percentile (ms)	p50 = 377.34, p75 = 466.18, p95 = 952.83, p99 = 1854.46

Запуск и ключевые метрики

Запуск: `dotnet run --project MyLoadTest.csproj` или из IDE.

Что смотрим в отчете NBomber:

- **RPS** (Requests per Second): Фактическая пропускная способность. Сравниваем с ожидаемой.
- **Latency** (Время отклика): Ключевые перцентили — p95 и p99. Если p99 = 2 сек, значит 1% пользователей ждал дольше 2 секунд.
- **Количество ошибок** и их тип (таймауты, некорректные ответы). Рост ошибок с ростом нагрузки — явный признак проблемы.
- **Data Transfer**: Не перегружаем ли мы сеть.



Отчёт и основные метрики (RPS, latency)

После запуска NBomber печатает результаты в консоль и генерирует HTML/txt-отчёты с таблицами по каждому сценарию и шагу; в отчёте есть блоки requests (all/ok/fail, RPS) и latency (min/mean/max, StdDev, p50/p75/p95/p99).

Типичные поля отчёта (по сценарию или шагу):

Метрика	Смысл
requests all / ok / fail	Общее количество запросов, успешно обработанных и завершившихся ошибкой.
RPS	Requests per second: среднее количество запросов в секунду.
latency min/mean/max	Минимальное, среднее и максимальное время ответа в миллисекундах.
latency p50/p75/p95/p99	Квантили времени ответа: медиана, 75-й, 95-й, 99-й перцентиль.

LIVE

Вопросы



если есть вопросы



если вопросов нет

Интеграция с OpenTelemetry

Диагностика dotnet-counters

Как работает интеграция с OpenTelemetry

OpenTelemetry собирает телеметрические данные (метрики, логи, трассировки) и отправляет их в системы мониторинга.

NBomber использует для этого пакет: [NBomber.Sinks.OpenTelemetry](#).

Существует два основных подхода к настройке:

- **Прямая отправка:** Метрики отправляются напрямую в бэкенд (например, Aspire Dashboard, Grafana).
- **Через коллектор:** Метрики сначала поступают в OpenTelemetry Collector, который затем перенаправляет их в один или несколько бэкендов для обработки и хранения.



Настройте синик (sink) в коде тест

```
using NBomber.CSharp;  
using NBomber.Sinks.OpenTelemetry;  
  
var scenario = // ... определение вашего сценария ...  
  
var stats = NBomberRunner  
    .RegisterScenarios(scenario)  
    .WithReportingSinks(  
        new OpenTelemetrySink(new OtlpExporterOptions  
        {  
            // Эндпоинт OTLP  
            Endpoint = new Uri("http://localhost:18889"),  
            // Протокол (Grpc или HttpProtobuf)  
            Protocol = OtlpExportProtocol.Grpc  
        })  
    )  
    .Run();
```



dotnet-counters

dotnet-counters — это кроссплатформенный инструмент командной строки для мониторинга производительности .NET-приложений в режиме реального времени. Он собирает метрики, публикуемые через API EventCounter или Meter, и идеально подходит для быстрой диагностики проблем с CPU, памятью, GC и другими компонентами без остановки приложения.

Работа с инструментом начинается с нескольких ключевых команд:

Команда	Назначение	Пример использования
monitor	Отображение обновляемых значений выбранных счетчиков.	<code>dotnet-counters monitor --process-id 1234 System.Runtime</code>
collect	Сбор и запись значений счетчиков в файл для последующего анализа.	<code>dotnet-counters collect --process-id 1234 --format csv -o report.csv</code>
list	Вывод списка доступных поставщиков счетчиков.	<code>dotnet-counters list</code>
ps	Поиск идентификаторов процессов (PID) запущенных .NET-приложений.	<code>dotnet-counters ps</code>



Установка dotnet-counters

Самый простой и рекомендуемый — установить его как глобальную .NET-утилиту:

```
dotnet tool install --global dotnet-counters
```

Проверка установки:

```
dotnet-counters --version
```

Если утилита уже установлена, её можно обновить до последней версии командой:

```
dotnet tool update --global dotnet-counters
```

Найти запущенные .NET-процессы:

```
dotnet-counters ps
```



Когда и как использовать

Когда применять:

- Для быстрой диагностики проблем на локальной машине или продакшене.
- Как часть CI/CD, чтобы собирать метрики во время нагрузочного тестирования (например, параллельно с NBomber).

Пример сценария:

1. Соберите базовые метрики приложения в состоянии покоя.
2. Запустите нагрузочный тест (например, с помощью NBomber).
3. Параллельно запустите `dotnet-counters collect`, чтобы записать метрики в файл.
4. Сравните результаты до и во время нагрузки, чтобы найти узкие места (например, резкий рост потребления памяти или частые сборки мусора).



Какие метрики смотреть

Самый важный поставщик — *System.Runtime*.

Он включает:

- **CPU Usage (%)**: загрузка процессора.
- **Allocation Rate (B/sec)**: скорость выделения памяти.
- **GC Heap Size (MB)**: размер управляемой кучи.
- **% Time in GC**: время, потраченное на сборку мусора.
- **Exception Count**: количество исключений.

Пример мониторинга нескольких счётчиков:

```
dotnet-counters monitor -p 12345 System.Runtime Microsoft.AspNetCore.Hosting
```



Пример скрипта для автоматизации

Для интеграции с нагрузочным тестированием можно использовать простой скрипт:

```
#!/bin/bash
# 1. Запустите приложение и получите его PID
PID=$(dotnet run & echo $!)

# 2. Начните сбор метрик в файл
dotnet-counters collect --process-id $PID --format csv -o metrics.csv
--duration 00:03:00 &

# 3. Запустите нагрузочный тест NBomber
dotnet run --project NBomberTestProject

# 4. Анализируйте CSV-файл с метриками
```



Анализ и корреляция данных. Комплексный подход

Проблема:

NBomber показывает ЧТО происходит (много ошибок, большая задержка), но не ПОЧЕМУ.

Решение: Триада наблюдения (Monitoring Triad):

- **Метрики приложения (dotnet-counters):**

В реальном времени смотрим на: % Time in GC, ThreadPool Queue Length, CPU Usage, Allocation Rate. Рост очереди потоков при высокой CPU — сигнал.

- **Трассировки (OpenTelemetry):**

Инструментируем код, чтобы видеть цепочку вызовов (Tracing). Где именно тратится время: в сетевом вызове, сериализации или блокировке (lock)?

- **Логи (Logs):**

Контекст ошибок.

Итог: Сопоставляем график latency из NBomber с графиком ThreadPool Queue Length из dotnet-counters. Если они растут синхронно — проблема в нехватке потоков или блокировках.



Промежуточные выводы

OpenTelemetry — это комплексная система сбора телеметрии для постоянного мониторинга.

Она требует настройки, но позволяет отправлять данные в централизованные хранилища (Prometheus, Grafana) и строить дашборды.

OpenTelemetry применяется для постоянного наблюдения за приложением.

dotnet-counters — это лёгкий ad-hoc инструмент для разработчиков.

Он не требует настройки в коде приложения, идеален для быстрой проверки.



Вопросы



если есть вопросы



если вопросов нет

Практический пример ТСР-кэш

LIVE

Подведём итоги занятия

Практические выводы

Закрепили на практике:

- Планировать тест: определять цели, сценарии, метрики успеха.
- Реализовывать его на современном .NET-стеке (NBomber).
- Анализировать результаты: находить не только пределы пропускной способности (RPS), но и точки деградации (p99 latency, ошибки).
- Проводить комплексную диагностику, связывая данные теста с метриками исполняющей среды.

Главный итог:

Нагрузочное тестирование — это не магия, а инженерная практика, которая делает ваши .NET-сервисы предсказуемыми и устойчивыми в продакшене.



Цели вебинара

К концу занятия вы сможете

1. Планировать и проводить нагрузочное тестирование для сетевых сервисов;
2. Использовать NBomber для генерации нагрузки;
3. Анализировать результаты тестов: RPS (запросы в секунду), время отклика (latency), количество ошибок;
4. Определять максимальную пропускную способность и точки отказа приложения.



Список материалов для изучения

- <https://sjinnovation.com/explore-various-testing-methods-software-comprehensive-guide>
- <https://jmeter.apache.org/>
- <https://gatling.io/>
- <https://k6.io/>
- <https://habr.com/ru/companies/ozontech/articles/662800/>
- <https://testengineer.ru/performance-testing-metrics/>
- <https://habr.com/ru/companies/otus/articles/675044/>
- <https://habr.com/ru/articles/664824/>
- <https://nbomber.com/>
- <https://www.nuget.org/packages/nbomber/>
- <https://antondevtips.com/blog/load-testing-microservices-with-csharp-and-nbomber>
- <https://www.nuget.org/packages/NBomber.Sinks.OpenTelemetry>
- <https://learn.microsoft.com/en-us/dotnet/core/diagnostics/dotnet-counters>
- <https://github.com/dotnet/diagnostics/blob/main/documentation/dotnet-counters-instructions.md>





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

